



Fleet Management Dashboard: Hiring Challenge

SDE-1 role, Peppermint Robotics

This is a hiring challenge for the **SDE-1** role at **Peppermint Robotics**. We are hiring full stack engineers, so this challenge is full stack: you build one small system end to end, from simulated robots, through a backend, to a dashboard in the browser, and you deploy it live so we can use it. This document tells you what the system must do and nothing about how to build it. The how is yours, and your choices are part of what we evaluate.

How this works

- **Deadline.** Send your submission within 3 days of receiving this package. We move fast, and part of what we are gauging is how quickly you can pick up an unfamiliar problem, use your tools well, and ship something honest. Cut scope deliberately if you must, and say what you cut; an honest, well reasoned gap scores better than a complete submission that cuts corners quietly. If something genuinely blocks you, tell us before the deadline, not after.
- **How to submit.** Email kautilya.boga@peppermintrobotics.com with a link to a git repository, or an archive of your source. Use this subject line, so your submission is not missed: `SDE-1 Challenge | <your name> | Full Stack`
- **AI tooling.** Use any AI tooling you are comfortable with, freely but carefully. Note in your README which parts you delegated to AI. AI use is not penalized; inability to explain any part of your own submission is.
- **Managed services.** Modern managed services (Supabase, Firebase, hosted message brokers, Railway, anything) are welcome wherever they fit, as long as the required deliverables still work as specified and you can explain what the service is doing for you, and what you would do if it disappeared.
- **Your work stays yours.** The data in this package is synthetic. Your code remains yours and you may keep it public in your portfolio. We use submissions only for evaluation.

The data

The package contains three files describing one fleet and site. Treat them as the data contract, not an asset to replay: they define the schema, the statuses, the coordinate system, and what plausible robot behavior looks like. Your simulator must produce data in this format, on this site; replaying the

recorded log itself is optional.

- `layout.png` is the site, as a plain image. The origin, $(0, 0)$, is the top left corner; x increases to the right and y increases downward, matching standard image coordinates. One pixel equals one unit of distance, so there is no scale conversion involved.
- `robots.json` is a roster of eight robots, each with a `robot_id`, a `robot_type`, and a starting position in the same coordinate system as the image.
- `events.jsonl` is a recorded log covering fifteen minutes, one JSON object per line: each robot reports its position, battery percentage, and status roughly every five seconds. A typical line looks like this:

```
{"t": 0, "robot_id": "r2", "x": 787.3, "y": 65.2, "status":  
"idle", "battery": 75.8}
```

`t` is seconds from the start of the window (0 to 900). `battery` is a percentage. `status` is one of `idle`, `active`, `on_mission`, `charging`, `blocked`, `error`, `maintenance`, or `offline`.

A couple of lines also carry an extra `task_event` key (`task_started` or `task_completed`). They are rare in this window and may be unpaired. Nothing is graded on them; surface them or ignore them, your call.

We deliberately do not define which statuses count as working or as needing attention, or what `active` means versus `on_mission`. Make a sensible call and be ready to defend it.

This log comes from a small scripted simulation on our end, so there is nothing hidden in it you need to reverse engineer.

The system you are building

This service will be used by an operator who is responsible for a fleet of robots that move around a site. Each robot reports a battery percentage, a position, and a status describing what it is currently doing. That data does not arrive as a file: each robot publishes it directly, live, and you own the whole path from the robots to the operator's browser. You may use any language, framework, protocol, or packaging.

1. Mock the robots

Run a simulated fleet that publishes each robot's `robot_id`, position, battery percentage, and status, in the format the data contract defines. Generate the fleet yourself: start from the roster in `robots.json` if you like, or invent your own robots. Plausibility counts and is graded: robots move continuously rather than teleporting, stay on the site, transition between statuses sensibly, and their batteries drain and charge like batteries. The recorded log shows you what plausible looks like.

Fleet size, update interval, and payload size must be configurable without code changes

(environment variables or a config file, documented in your README). We will turn these up when we test: think hundreds or thousands of robots, faster updates, larger payloads. Make at least fleet size and update interval adjustable on your deployed instance too, without a redeploy, through a control you document in your README, protected sensibly. How the robots get their data to your backend is entirely your call: MQTT, a message queue, gRPC, plain sockets, HTTP, anything that fits a producer and consumer split.

2. Build the backend

Run a backend service that ingests every robot's updates and maintains the fleet's current state.

Ingestion must not block or lock up as the fleet grows: a burst of updates or a slow consumer should not stall the pipeline. Expose the state to consumers through whatever mechanism you are ready to defend: WebSocket, server sent events, gRPC streaming, REST polling, anything. If you expose more than one surface, they must agree with each other.

Real deployments have flaky networks: a robot's connection can drop and reconnect, and so can a dashboard's. Your design should account for that rather than assume a clean, always on connection.

3. Build the dashboard

Build the frontend the operator actually uses, driven by your backend's live feed (not by reading `events.jsonl` in the browser). The operator needs to be able to:

1. **See the site and the robots on it, as they move**, legibly, at any fleet size your simulator can produce.
2. **See how things are trending** over time, for instance the fraction of the fleet active. Show at least one such trend, with controls to zoom or change the window. A single current value readout does not count, and neither does a static chart.
3. **Find a specific robot, or the ones that need attention**, and see enough about it to decide what to do next.

We judge the dashboard as a product, not a demo: layout, readability, and behavior under load all count. A dashboard that looks impressive with eight robots and becomes unusable with eight hundred has missed the point.

4. Deploy it live

Include working links to your deployed system: the dashboard, and whatever backend surface it consumes, publicly reachable, with the deployed dashboard talking to the deployed backend. List the URLs at the top of your README. A free hosting tier is enough; cold starts are fine if you note them. Your endpoints will be on the public internet: maintain basic security, and expect us to look. Verify your links in a private browser window before submitting.

Optional stretch goal

If you have headroom left: persist the fleet history and expose it as `GET /robots/history/{robot_id}` with a time range. Any store is fine (SQLite, Postgres, Supabase,

anything); one sentence on why you chose it. Optional means optional.

Findings

Send a `FINDINGS.md` written in your own words and your own structure. There is no template; what you choose to report is itself part of the evaluation. At a minimum it must cover:

1. **The real tradeoffs you made.** Your transport and exposure choices, what you rejected and why, and what your choice costs you.
2. **Where your system degrades as the fleet grows**, with the numbers you actually observed using your own configuration knobs, and what limits it first.
3. **What you cut**, and what you would build next.

Findings that could have been written without running the system score poorly.

Architecture

Send an `ARCHITECTURE.md` that shows how your system is connected and how data moves through it: a diagram (hand drawn and photographed is fine) plus a short walk from a robot publishing an event to a pixel changing on the dashboard. Cover what happens when things go wrong: a robot dies mid task, updates arrive late or out of order, a dashboard client drops and reconnects. Finish with what you would change first if the fleet grew another ten times.

Submitting your work

Your live system is the submission; everything else supports it. We open your URLs and use the product first, before we read a line of code, so lead your email with them: the dashboard, and the backend surface it consumes.

Alongside it, send your source (git repository or archive), a `README.md` (live URLs at the top, your configuration knobs and how to work the live controls, local run steps for a Linux machine, AI delegation notes), `FINDINGS.md`, `ARCHITECTURE.md`, and a couple of tests for the part you found trickiest. Packaging is your call (Docker, plain scripts, a Makefile, anything).

Checklist: live dashboard and backend URLs • source repo or archive • `README.md` (live URLs, knob controls, run steps for Linux, AI delegation notes) • `FINDINGS.md` • `ARCHITECTURE.md` • tests for the trickiest part

That is everything. If anything in this document is unclear, write to kautilya.boga@peppermintrobotics.com and ask; a good clarifying question costs you nothing. We look forward to seeing what you build.